

#QUESTION 1

create variable 'name' and 'batch'.Next,assign 'Amrita Vishwa Vidyapeetham' and '2025' to be variables.Write the code for the following output:

```
In [1]: name='Amrita Vishwa Vidyapeetham'
batch='2025'
print("my college's name is",name,"and my batch is",batch)
```

my college's name is Amrita Vishwa Vidyapeetham and my batch is 2025

2. Get your name and age using the input function and print the following output: Your name is _____ and your age is _____

```
In [1]: name='shubruh'
age=17
print('my name is',name,'and my age is',age)
```

my name is shubruh and my age is 17

3. Create a variable 'rupees', assign 4500 and get the output: I don't have 4500 rupees in my wallet. Next, assign 3000 to the same variable and then print: But I have "3000" rupees

```
In [4]: rupees='4500'
print("i don't have",rupees,"rupees in my wallet")
```

i don't have 4500 rupees in my wallet

```
In [20]: rupees="3000"
print('But i have "',rupees,'" rupees',sep="")
```

But i have "3000" rupees

4. Write a code for printing the roots of $x^2 + 4x + 3 = 0$

```
In [27]: import math as m
a=1
b=4
c=3
r1=(-b+m.sqrt(b**2-4*a*c))/2*a
r2=(-b-m.sqrt(b**2-4*a*c))/2*a
r1,r2
```

Out[27]: (-1.0, -3.0)

5. Write a program that will convert degrees Celsius to degrees Fahrenheit

```
In [1]: c=float(input('degree celcius'))  
f=9/5*c+32  
print('degree fareheit is',f)
```

```
degree celcius45  
degree fareheit is 113.0
```

6. Write a program to find the sum of n natural numbers. (n will be input)

```
In [7]: n=int(input('natural numbers:'))  
s=(n*(n+1))/2  
print('sum of n natural numbers',s)
```

```
natural numbers:10  
sum of n natural numbers 55.0
```

7. Write a Python code to print the speed if a runner completes 750 m in two and a half minute

```
In [8]: d=750  
tmin=2.5  
tsec=2.5*60  
s=d/tsec  
print('speed of the runner is',s)
```

```
speed of the runner is 5.0
```

8. Write Python code for the following programs; use an input function to get the needed values: a. Adding two numbers.

```
In [2]: a=int(input('first number:'))  
b=int(input('second number:'))  
print(a+b)
```

```
first number:5  
second number:5  
10
```

b. Multiplying two numbers.

```
In [3]: a=int(input('first number:'))  
b=int(input('second number:'))  
print(a*b)
```

```
first number:5  
second number:5  
25
```

c. Area of the square.

```
In [4]: s=int(input('side of the square:'))
A=s**2
print('area of the square of side is',A)
```

side of the square:5
area of the square of side is 25

d. Area of the rectangle

```
In [5]: l=int(input('length of the rectangle:'))
b=int(input('breadth of the rectangle:'))
A=l*b
print('area of the rectangle is',A)
```

length of the rectangle:5
breadth of the rectangle:3
area of the rectangle is 15

e. Area of the circle.

```
In [6]: import math
r=float(input('radius of the circle:'))
A=math.pi*(r**2)
print('area of the circle is',A)
```

radius of the circle:5
area of the circle is 78.53981633974483

f. The average of four numbers.

```
In [7]: a=float(input('first number:'))
b=float(input('second number:'))
c=float(input('third number:'))
d=float(input('fourth number:'))
A=(a+b+c+d)/4
print('average of four numbers is',A)
```

first number:1
second number:2
third number:3
fourth number:4
average of four numbers is 2.5

g. Calculate the arc length. (Length of an Arc = $\theta \times r$, where θ is in radians. Length of an Arc = $\theta \times (\pi/180) \times r$, where θ is in degrees.

```
In [8]: import math
r=float(input('radius of the arc:'))
theta=float(input('angle of the arc:'))
l=theta*r
L=theta*(math.pi/180)*r
print('length of arc in radians is',l,'\nlength of arc in degrees is',L)
```

```
radius of the arc:5
angle of the arc:30
length of arc in radians is 150.0
length of arc in degrees is 2.617993877991494
```

h. Calculate the surface volume and area of a sphere. (Surface area of the sphere = $4\pi r^2$, Volume of a sphere = $\frac{4}{3} \times \pi r^3$)

```
In [9]: import math
r=float(input('radius of sphere:'))
A=4*math.pi*(r**2)
V=(4/3)*math.pi*(r**3)
print('surface area of the sphere is',A,'\nvolume of the sphere is',V)
```

```
radius of sphere:5
surface area of the sphere is 314.1592653589793
volume of the sphere is 523.5987755982989
```

i. surface volume and area of a cylinder. (Surface area of a cylinder is $A = 2\pi rh + 2\pi r^2$, where r = radius, h = height. Volume of a cylinder is $V = \pi r^2 h$, where r = radius, h = height.

```
In [11]: import math
r=float(input('radius of the cylinder:'))
h=float(input('height of the cylinder:'))
A=(2*math.pi*r*h)+(2*math.pi*(r**2))
V=math.pi*(r**2)*h
print('surface area of the cylinder is',A,'\nvolume of the cylinder is',V)
```

```
radius of the cylinder:5
height of the cylinder:10
surface area of the cylinder is 471.23889803846896
volume of the cylinder is 785.3981633974483
```

9. Calculate the expressions:

- $2.3(3x+(6y+z^2)/3)$
- Get x , y , z using the input function.

```
In [16]: x=float(input('first variable:'))
y=float(input('second variable:'))
z=float(input('third variable:'))
expression=2.3*(3*x+(6*y+z**2)/3)
print('the value of given expression is',expression)
```

```
first variable:1
second variable:2
third variable:3
the value of given expression is 23.0
```

b. $\sqrt{5((a^3)^b + 2ac - c)}$ Get a, b, c using the input function.

```
In [18]: import math
a=float(input('first variable:'))
b=float(input('second variable:'))
c=float(input('third variable:'))
expression=math.sqrt(5)*(((a**3)**(b**2))+2*a*c-c)
print('the value of given expression is',expression)
```

```
first variable:1
second variable:2
third variable:3
the value of given expression is 8.944271909999916
```

10. Define 2 complex numbers x and y. Write Python programs for the following: a. Add, subtract, multiply, and divide two complex numbers

```
In [19]: a=float(input('real part of first number:'))
b=float(input('imaginary part of first number:'))
c=float(input('real part of second number:'))
d=float(input('imaginary part of second number:'))
x=complex(a,b)
y=complex(c,d)
print('sum=',x+y)
print('difference=',x-y)
print('product=',x*y)
print('divition=',x/y)
```

```
real part of first number:1
imaginary part of first number:2
real part of second number:3
imaginary part of second number:4
sum= (3+2j)
difference= (-1+2j)
product= (2+4j)
divition= (0.5+1j)
```

b. Print the complex conjugate, real and imaginary parts of a complex number

```
In [22]: print(x.conjugate())
print(x.real)
print(x.imag)
```

```
(1-2j)
1.0
2.0
```

11. Write a Python program to convert rectangular coordinates to polar coordinates in the first quadrant. (The polar coordinates of (a, b) is (r,theta) , where $r = \sqrt{a^2 + b^2}$, $\theta = \tan^{-1}(b/a)$)

```
In [29]: import math
a=float(input('first variable:'))
b=float(input('second variable:'))
r=math.sqrt(a**2+b**2)
theta=math.atan(b/a)
print('polar coordinates are (',r,',',theta,')')
```

```
first variable:3
second variable:4
polar coordinates are ( 5.0 , 0.9272952180016122 )
```

12. Write a program to find the following expression. (x is given as input) (a) $\cos(\sin x^2)$
(b) $\sec(\cot(\sqrt{e^x}))$ (c) $\log_{10}(\sin 2x)$ (d) $\cos^{-1}(x) + \log_5(x + 1)$

```
In [30]: import math
x=float(input('variable:'))
print(math.cos(math.sin(x**2)))
```

```
variable:5
0.9912542848596704
```

```
In [35]: import math
print(1/math.cos(1/math.tan(math.e**x)))
```

```
2.0300057070672386
```

```
In [47]: import math
print(math.log10(math.sin(math.pi/3)))
```

```
-0.06246936830415
```

In [41]: `help(math)`

Help on built-in module math:

NAME

math

DESCRIPTION

This module provides access to the mathematical functions defined by the C standard.

FUNCTIONS

`acos(x, /)`

Return the arc cosine (measured in radians) of x.

The result is between 0 and pi.

`acosh(x, /)`

Return the inverse hyperbolic cosine of x.

`asin(x, /)`

Return the arc sine (measured in radians) of x.

The result is between -pi/2 and pi/2.

`asinh(x, /)`

Return the inverse hyperbolic sine of x.

`atan(x, /)`

Return the arc tangent (measured in radians) of x.

The result is between -pi/2 and pi/2.

`atan2(y, x, /)`

Return the arc tangent (measured in radians) of y/x.

Unlike `atan(y/x)`, the signs of both x and y are considered.

`atanh(x, /)`

Return the inverse hyperbolic tangent of x.

`cbrt(x, /)`

Return the cube root of x.

`ceil(x, /)`

Return the ceiling of x as an Integral.

This is the smallest integer $\geq x$.

`comb(n, k, /)`

Number of ways to choose k items from n items without repetition and without order.

Evaluates to $n! / (k! * (n - k)!)$ when $k \leq n$ and evaluates to zero when $k > n$.

Also called the binomial coefficient because it is equivalent to the coefficient of k-th term in polynomial expansion of the expression $(1 + x)^n$.

Raises `TypeError` if either of the arguments are not integers.
Raises `ValueError` if either of the arguments are negative.

`copysign(x, y, /)`

Return a float with the magnitude (absolute value) of x but the sign of y.

On platforms that support signed zeros, `copysign(1.0, -0.0)` returns `-1.0`.

`cos(x, /)`

Return the cosine of x (measured in radians).

`cosh(x, /)`

Return the hyperbolic cosine of x.

`degrees(x, /)`

Convert angle x from radians to degrees.

`dist(p, q, /)`

Return the Euclidean distance between two points p and q.

The points should be specified as sequences (or iterables) of coordinates. Both inputs must have the same dimension.

Roughly equivalent to:

```
sqrt(sum((px - qx) ** 2.0 for px, qx in zip(p, q)))
```

`erf(x, /)`

Error function at x.

`erfc(x, /)`

Complementary error function at x.

`exp(x, /)`

Return e raised to the power of x.

`exp2(x, /)`

Return 2 raised to the power of x.

`expm1(x, /)`

Return $\exp(x)-1$.

This function avoids the loss of precision involved in the direct evaluation of $\exp(x)-1$ for small x.

`fabs(x, /)`

Return the absolute value of the float x.

`factorial(n, /)`

Find n!.

Raise a `ValueError` if x is negative or non-integral.

`floor(x, /)`

Return the floor of x as an `Integral`.

This is the largest integer $\leq x$.

`fmod(x, y, /)`

Return `fmod(x, y)`, according to platform C.

$x \% y$ may differ.

`frexp(x, /)`

Return the mantissa and exponent of x, as pair (m, e).

m is a float and e is an int, such that $x = m * 2.^e$.

If x is 0, m and e are both 0. Else $0.5 \leq \text{abs}(m) < 1.0$.

`fsum(seq, /)`

Return an accurate floating point sum of values in the iterable se

q.

Assumes IEEE-754 floating point arithmetic.

`gamma(x, /)`

Gamma function at x.

`gcd(*integers)`

Greatest Common Divisor.

`hypot(...)`

`hypot(*coordinates) -> value`

Multidimensional Euclidean distance from the origin to a point.

Roughly equivalent to:

`sqrt(sum(x**2 for x in coordinates))`

For a two dimensional point (x, y), gives the hypotenuse using the Pythagorean theorem: `sqrt(x*x + y*y)`.

For example, the hypotenuse of a 3/4/5 right triangle is:

```
>>> hypot(3.0, 4.0)
```

```
5.0
```

`isclose(a, b, *, rel_tol=1e-09, abs_tol=0.0)`

Determine whether two floating point numbers are close in value.

`rel_tol`

maximum difference for being considered "close", relative to t

he

magnitude of the input values

`abs_tol`

maximum difference for being considered "close", regardless of

the

magnitude of the input values

Return True if a is close in value to b, and False otherwise.

For the values to be considered close, the difference between them must be smaller than at least one of the tolerances.

-inf, inf and NaN behave similarly to the IEEE 754 Standard. That is, NaN is not close to anything, even itself. inf and -inf are only close to themselves.

`isfinite(x, /)`

Return True if x is neither an infinity nor a NaN, and False otherwise.

`isinf(x, /)`

Return True if x is a positive or negative infinity, and False oth

erwise.

`isnan(x, /)`

Return True if x is a NaN (not a number), and False otherwise.

`isqrt(n, /)`

Return the integer part of the square root of the input.

`lcm(*integers)`

Least Common Multiple.

`ldexp(x, i, /)`

Return $x * (2^{**i})$.

This is essentially the inverse of `frexp()`.

`lgamma(x, /)`

Natural logarithm of absolute value of Gamma function at x.

`log(...)`

`log(x, [base=math.e])`

Return the logarithm of x to the given base.

If the base not specified, returns the natural logarithm (base e) of x.

`log10(x, /)`

Return the base 10 logarithm of x.

`log1p(x, /)`

Return the natural logarithm of $1+x$ (base e).

The result is computed in a way which is accurate for x near zero.

`log2(x, /)`

Return the base 2 logarithm of x.

`modf(x, /)`

Return the fractional and integer parts of x.

Both results carry the sign of x and are floats.

`nextafter(x, y, /)`

Return the next floating-point value after x towards y.

`perm(n, k=None, /)`

Number of ways to choose k items from n items without repetition and with order.

Evaluates to $n! / (n - k)!$ when $k \leq n$ and evaluates to zero when $k > n$.

If k is not specified or is None, then k defaults to n and the function returns n!.

Raises `TypeError` if either of the arguments are not integers.
Raises `ValueError` if either of the arguments are negative.

`pow(x, y, /)`

Return x^{**y} (x to the power of y).

`prod(iterable, /, *, start=1)`

Calculate the product of all the elements in the input iterable.

The default start value for the product is 1.

When the iterable is empty, return the start value. This function is intended specifically for use with numeric values and may reject non-numeric types.

`radians(x, /)`

Convert angle x from degrees to radians.

`remainder(x, y, /)`

Difference between x and the closest integer multiple of y.

Return $x - n*y$ where $n*y$ is the closest integer multiple of y.

In the case where x is exactly halfway between two multiples of y, the nearest even value of n is used. The result is always exact.

`sin(x, /)`

Return the sine of x (measured in radians).

`sinh(x, /)`

Return the hyperbolic sine of x.

`sqrt(x, /)`

Return the square root of x.

`tan(x, /)`

Return the tangent of x (measured in radians).

`tanh(x, /)`

Return the hyperbolic tangent of x.

`trunc(x, /)`

Truncates the Real x to the nearest Integral toward 0.

Uses the `__trunc__` magic method.

`ulp(x, /)`

Return the value of the least significant bit of the float x.

DATA

`e = 2.718281828459045`

`inf = inf`

`nan = nan`

`pi = 3.141592653589793`

`tau = 6.283185307179586`

FILE

(built-in)

In [64]: `print(math.acos(0)+math.log(x,5))`

2.5707963267948966

In [61]: `math.acos(0)`

Out[61]: 1.5707963267948966

In [62]: `math.log(x,5)`

Out[62]: 1.0

In []: